

Objectif

Décrire la couche **Application / Controllers**, qui coordonne :

- les actions venant de l'UI,
- la logique métier du **Domain** (sessions, exercices),
- la **Persistence** (chargement et sauvegarde via repositories).

Ce diagramme ne décrit pas l'UI ni les détails SQL : uniquement les **dépendances structurantes**.

Diagramme

```
%%{init: {"class": {"hideEmptyMembersBox": true}} }%%
classDiagram

namespace Application {
    class HomeController
    class SessionController
    class StatsController
    class SettingsController
}

namespace Domain {
    class Session
    class WordSession
    class SentenceSession
    class Exercise
    class TemplateType
}

%% Domain inheritance
Session <|-- WordSession
Session <|-- SentenceSession

%% Application -> Domain
HomeController --> SessionController : starts sessions
SessionController --> Session : manages
SessionController --> Exercise : produces/consumes
SessionController --> TemplateType : selects template

%% Application -> Persistence
SessionController --> Repository : **load**<br/> words, groups, chapters
SessionController --> Repository : **load/save** SRS

StatsController --> Repository: read
```

Lecture du diagramme

Controllers (Application)

- **HomeController** : logique d'entrée (démarrer une session, navigation logique côté app).
- **SessionController** : orchestre le déroulement d'une session (sélection du contenu, génération d'exercices, soumission des réponses).
- **StatsController** : calcule et expose les statistiques à partir des données persistées, sans dépendre des sessions ou du runtime d'exercices.
- **SettingsController** : expose et modifie la configuration (ex: paramètres SRS).

Domain

- **Session** est une abstraction : **WordSession** et **SentenceSession** sont deux spécialisations.
- **Exercise** est un objet **runtime** utilisé pendant une session.
- **TemplateType** identifie le template de rendu choisi (implémentation côté UI/Presentation).

Persistence

Les controllers accèdent aux données via des **repositories** :

- contenu (**WordRepository**, **SentenceGroupRepository**, **ChapterRepository**)
- progression/config (**SrsRepository**)

Le SQL, le mapping DB, et les tables sont confinés au diagramme "Persistence".

Règles d'architecture

PROF

- L'UI appelle les controllers ; elle n'appelle jamais directement le Domain ou la Persistence.
- Le **StatsController** ne dépend pas du runtime (sessions/exercices), uniquement des repositories.
- Le **SessionController** orchestre les sessions et délègue la persistance aux repositories (aucun SQL ici).
- Le Domain reste indépendant de la couche UI Flutter.

Note sur le cycle de vie des Controllers

Les **Controllers** sont des objets **long-vivants**, créés lors de l'initialisation de l'application et partagés entre les écrans.

- Ils conservent les dépendances (repositories, configuration).
- Ils **ne représentent pas** une session en cours.
- Ils créent et détruisent les **Sessions** (**WordSession**, **SentenceSession**) à la demande.

Les **Sessions** sont des objets **éphémères**, limités à la durée d'une session d'apprentissage.

Ce découplage permet d'enchaîner plusieurs sessions sans recréer les Controllers et garantit une gestion claire du cycle de vie.